

Step 11 - Talend ETL Patterns

Phase D · tMap Design · Reconciliation · Tool-Agnostic Fluency · ~20 min

Goal: Talend pattern fluency via two design artifacts (free Talend was discontinued Jan 2024, so design docs replace installs - and often impress interviewers more). Artifacts: talend/pii_masking_rules.md + talend/reconciliation_design.md

LEARN

Talend in one breath: a Job is a visual pipeline of components wired left-to-right; data flows through rows, control through triggers (OnSubjobOk, Run If); everything compiles to Java.

Component	Role	MapleBank use
tFileInputDelimited	Read CSV with explicit schema	Core banking customer extract
tMap	Transform + filter + route to multiple outputs	PII masking, valid/reject routing
tFileOutputDelimited	Write CSV	Staging + reject files
tLogCatcher	Catch any runtime exception	Error file for ops
tDie / tAssert	Kill job / pre-flight checks	Halt EOD on bad input

The translation table - your secret weapon (you already built all of this in PySpark):

Talend	Your PySpark equivalent
tMap expressions	withColumn + F.when
tMap output filters	filter / valid-reject split
tMap lookup	join (broadcast for small dims)
Context variables	dbutils.widgets / ADF parameters
tLogCatcher	try/except + quality report
Reject flow + reason code	your reject_reason pattern

LAB — Artifact 1: talend/pii_masking_rules.md

Document the job design (the same masking your Step 6 Silver layer performs):

```

# Job_Customer_PII_Mask
Flow: tFileInputDelimited (explicit schema)
  -> tMap (mask + validate + route)
      |- valid_rows -> customer_staging_YYYYMMDD.csv
      |- reject_rows -> customer_rejects_YYYYMMDD.csv (with reason code)
tLogCatcher -> job_errors_YYYYMMDD.csv

Masking rules (PIPEDA):
  sin          -> last 3 digits + SHA-256 hash col   (joins w/o exposure)
  account_number -> last 3 digits                     (CSR-recognizable)
  postal_code   -> FSA (first 3 chars)                 (~7,000 people)
  email         -> first char + domain
  phone        -> last 4 digits
  street_address -> DROP entirely

tMap expressions (Java):
  sin_masked:  "****-***-" + row1.sin.substring(8,11)
  postal_fsa:  row1.postal_code.substring(0,3).toUpperCase()

Validation filter (valid output):
  row1.sin != null && row1.sin.matches("[0-9]{3}-[0-9]{3}-[0-9]{3}")
  && row1.province.matches("ON|QC|BC|AB|MB|SK|NS|NB|NL|PE|YT|NT|NU")
  && row1.postal_code.matches("[A-Z][0-9][A-Z] ?[0-9][A-Z][0-9]")
  && row1.transit_number.matches("[0-9]{5}")

Reject reason (ternary chain):
  row1.sin == null ? "SIN_NULL"
  : !row1.sin.matches(...) ? "SIN_FORMAT_INVALID"
  : !row1.province.matches(...) ? "PROVINCE_INVALID"
  : "POSTAL_CODE_INVALID"

Context vars: INPUT_PATH / OUTPUT_PATH / BUSINESS_DATE per DEV/UAT/PROD
Rejects retained 7 years (OSFI).

```

LAB — Artifact 2: talend/reconciliation_design.md

```
# Job_Recon_Transactions_vs_GL
Purpose: nightly check that warehouse totals match the General Ledger.
A wrong dashboard in a bank is an OSFI incident -> EOD halts on failure.

Inputs joined on branch_id + transaction_type + business_date:
  A) txn_totals.csv (warehouse aggregates, from Gold daily_branch_summary)
  B) gl_totals.csv (authoritative GL from core banking)
variance_cad = txn_total - gl_total

Three-tier outcome:
  MATCH    abs(var) < $0.01 AND counts equal -> match file (audit proof)
  VARIANCE abs(var) <= $1.00 (context tol.) -> log + notify DQ team
  FAILURE  abs(var) > $1.00 OR count mismatch-> alert + System.exit(1)

Why System.exit(1): non-zero exit -> orchestrator (TAC or ADF) marks the
job FAILED -> master EOD halts -> Gold never builds on bad data ->
nothing wrong reaches FINTRAC reports. Failing loudly is the design goal.

Error handling: tLogCatcher (exceptions), tAssert (input rows >= 1000),
tDie (fatal stop), tWarn (within-tolerance logging).
Retention: match/variance/failure files 7 years (OSFI) - match files are
PROOF reconciliation ran and passed on every date.

SQL equivalent:
SELECT t.branch_id, t.transaction_type,
       t.total_cad - g.gl_cad AS variance,
       CASE WHEN ABS(t.total_cad-g.gl_cad) < 0.01 THEN 'MATCH'
            WHEN ABS(t.total_cad-g.gl_cad) <= 1.00 THEN 'VARIANCE'
            ELSE 'FAILURE' END AS recon_status
FROM txn_totals t FULL OUTER JOIN gl_totals g
  ON t.branch_id=g.branch_id AND t.transaction_type=g.transaction_type
  AND t.business_date=g.business_date;
```

COMMIT + INTERVIEW SOUNDBITE

Both files to talend/ on GitHub.

"I design masking and reconciliation tool-agnostically - in Talend they are tMap expressions with valid/reject routing and context variables; in our Databricks stack the identical logic lives in a shared PySpark utils notebook. The pattern matters more than the tool: explicit validation, reject flows with reason codes, parameterized promotion, and reconciliation that halts the EOD loudly rather than letting bad data reach FINTRAC."